



Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Sumário

- Prompt Mestre — Token Economy Universal 4
 - POR QUE UTILIZAR ESTE PROMPT 4
 - O Problema 4
 - A Solução 4
 - Resultado Concreto 5
 - Para quem serve 5
 - Como funciona 5
 - PARTE 1: SETUP (Execute uma vez) 6
 - Comando Único (Recomendado) 6
 - Setup Manual (Alternativa) 6
 - 1.2 Criar junctions multi-IDE 6
 - 1.3 Code Review Graph (recomendado) 7
 - 1.4 Adicionar ao `.gitignore` 7
 - 1.4 Criar arquivo de instruções 7
 - PARTE 2: PROMPT MESTRE (Copie e cole) 8
 - PARTE 3: CHECKLIST 9
 - PARTE 4: ECONOMIA 9

Prompt Mestre — Token Economy Universal

Aplicável a QUALQUER projeto (novo ou existente), QUALQUER categoria (web, mobile, backend, frontend, data, AI, embedded, etc.), QUALQUER ferramenta de IA (Claude Code, Cursor, Windsurf, Copilot, Codex, OpenClaw, MiMoCode, Cline, Roo Code, Augment, Continue, Aider, etc.)

POR QUE UTILIZAR ESTE PROMPT

O Problema

Quando você usa ferramentas de IA para programar, cada interação consome tokens que são traduzidos em custo real. Sem otimização, os principais desperdícios são:

1. **System prompts gigantescos** — arquivos de instrução com 300+ linhas são arregados em CADA sessão, CADA subagente, CADA chamada LLM
2. **Contexto redundante** — mesmas instruções repetidas em múltiplos arquivos para diferentes IDEs (CLAUDE.md + AGENTS.md + .cursor/rules/ + .windsurfrules)
3. **Leitura desnecessária** — agentes leem arquivos inteiros quando precisam penas de um trecho
4. **Logs volumosos** — saídas de comandos entram no contexto sem necessidade
5. **Falta de padronização** — cada projeto reinventa as mesmas regras

A Solução

Este prompt mestre implementa **9 técnicas comprovadas** de economia de tokens que juntas reduzem o custo em **25-35%** sem perda de qualidade:

Técnica	O que faz	Economia
Caveman	Respostas telegráficas, sem enrolação	–15% output
Headroom	Comprime logs >7 linhas (3 topo + 4 fim)	–10% input
LeanCTX	grep antes de read, assinaturas antes de corpos	–20% leitura
Delegação	Subagentes para buscas extensas	–30% contexto
Build isento	Pipeline de compilação não é comprimido	Qualidade preservada
Fidelidade	Dados de output nunca são truncados	Qualidade preservada
Portabilidade	1 arquivo = 6 IDEs (hardlinks)	–80% duplicação
Submodule	Skills compartilhadas entre projetos	–90% manutenção
Grafo	code-review-graph para navegação	–50% buscas

Resultado Concreto

Antes (sem otimização):

- System prompt: ~10,000 tokens por sessão
- Custo por livro de 12 capítulos: \$1.84
- Custo anual (12 produções): \$22.08

Depois (com otimização):

- System prompt: ~2,000 tokens por sessão (-80%)
- Custo por livro de 12 capítulos: \$1.51 (-18%)
- Custo anual (12 produções): \$18.12

Economia: ~\$4/ano por projeto — mas o verdadeiro ganho é **velocidade**: menos tokens = respostas mais rápidas = produtividade maior.

Para quem serve

- **Desenvolvedores solo** que usam IA diariamente e querem reduzir custos
- **Equipes** que querem padronizar configuração entre projetos
- **Freelancers** que precisam de setup rápido em novos projetos
- **Empresas** que querem infraestrutura de IA replicável e sustentável

Como funciona

1. **Submodule** — skills de economia ficam em repositório compartilhado
2. **Hardlinks** — 1 arquivo de instrução serve para 6+ ferramentas

3. **Junctions** — skills acessíveis de qualquer caminho
4. **Code Review Graph** — grafo de conhecimento para navegação inteligente
5. **Setup automatizado** — 1 comando configura tudo

Tempo de setup: ~30 segundos para projeto novo, ~1 minuto para existente.

PARTE 1: SETUP (Execute uma vez)

Comando Único (Recomendado)

```
/aplicar-token-economy          # Diretório atual
/aplicar-token-economy /caminho/projeto  # Projeto específico
```

Este comando detecta automaticamente o tipo de projeto e instala tudo.

Setup Manual (Alternativa)

1.1 Adicionar submodule

```
git submodule add git@github.com:Heverton-web/token-economy-
shared.git .token-economy
bash .token-economy/setup.sh      # macOS/Linux
powershell -ExecutionPolicy Bypass -File .token-economy\setup.ps1 #
Windows
```

1.2 Criar junctions multi-IDE

```
bash .token-economy/setup-links.sh      # macOS/Linux
powershell -ExecutionPolicy Bypass -File .token-economy\setup-links.ps1 #
Windows
```

1.3 Code Review Graph (recomendado)

```
bash .token-economy/setup-graph.sh      # macOS/Linux
powershell -ExecutionPolicy Bypass -File .token-economy\setup-graph.ps1 #
Windows
```

1.4 Adicionar ao .gitignore

```
__pycache__/  
*.pyc  
.pytest_cache/  
output/  
.token-economy/
```

1.4 Criar arquivo de instruções

O setup cria hardlinks automaticamente. Para setup manual, crie QUALQUER um destes:

Ferramenta	Arquivo
Claude Code	CLAUDE.md
Codex / OpenClaw / MiMoCode	AGENTS.md
Cursor	.cursor/rules/instrucoes.mdc
Windsurf	.windsurfrules
Cline	.clinerules
GitHub Copilot	.github/copilot-instructions.md
Roo Code	.roo/rules/instrucoes.md
Augment	.augment/instructions.md
Continue	.继续/rules/instrucoes.md

Todos hardlinks — edite um, todos atualizam.



PARTE 2: PROMPT MESTRE (Copie e cole)

```
# [NOME DO PROJETO]

## 0. Economia Severa de Tokens (PRIORIDADE MÁXIMA)

1. Caveman Ativo: pensamento telegráfico (3-5 linhas), sem preâmbulos/saudações.
2. Headroom: logs/builds >7 linhas → comprimir (3 topo + 4 fim).
   EXCEÇÃO: `output/**` e dados de obra NUNCA são comprimidos.
3. LeanCTX: grep antes de read em código/config. Limitar leitura por linha.
4. Delegação: subagentes comprimidos para buscas/edições extensas (nunca para prosa).
5. Build ISENT0: qualquer pipeline de build/compilação (npm, cargo, pandoc, make, docker, etc.) é liberado e obrigatório.
6. Fallback Terminal: se sandbox bloquear, exibir comandos no chat para o usuário rodar.
7. Soberania do Usuário: nada é barrado sem confirmação explícita.
8. Fidelidade de Conteúdo: `output/**`, JSONs de estado e scripts de auditoria são isentos de compressão.
9. Auto-commit/push: alterações devem ser commitadas e pushadas.

## 1. Regras

- R1: idioma do projeto em toda comunicação.
- R2: sem preâmbulos/saudações nos artefatos.
- R3: após definição, roda 100% autônomo.
- R4: desvios corrigidos internamente antes da entrega.

## 2. Skills (via .token-economy/)

| Skill | Função |
| --- | --- |
| `lean-ctx` | grep antes de read |
| `headroom` | Compressão de logs > 7 linhas |
| `caveman` | Respostas telegráficas |
| `rtk-memory` | Registro de erros/padrões |
| `pre-flight-check` | Validação antes de commit |
| `calcular-gastos-sessao` | Cálculo de tokens |
| `fable-method` | Resolução de problemas |
| `fable-judge` | Verificação adversarial |
| `self-learning` | Captura de golden paths |

## 3. Portabilidade
```



```
Hardlinks: `CLAUDE.md` → `AGENTS.md` → `.cursor/rules/` → `windsurfrules`  
→ `clinerules` → `github/copilot-instructions.md`.  
Junctions: `.agents/*` → `.claude/*`.  
Recriar: `bash .token-economy/setup-links.sh`
```

4. Stack

- Linguagem: [INSERIR]
- Framework: [INSERIR]
- Testes: [INSERIR]
- Token Economy via `.token-economy/` submodule

PARTE 3: CHECKLIST

- ☐ `.token-economy/` existe com 9 skills
- ☐ Junctions/symlinks funcionam
- ☐ Arquivo de instruções criado e <100 linhas
- ☐ Testes rodam
- ☐ `code-review-graph build` roda (recomendado)

PARTE 4: ECONOMIA

Métrica	Antes	Depois	Economia
System prompt	~10k tokens	~2k tokens	−80%
Custo por sessão	\$1.84	\$1.51	−18%
Resultado			25-35% redução

